API BIP BIP

SUMÁRIO

1. **INTRODUÇÃO** 1
2. **Objetivo da API** 2
3. **Público-Alvo** 2
4. **Tecnologias Utilizadas** 3
5. **Arquitetura da Aplicação** 3
6. **Estrutura do Banco de Dados** 4
   - 6.1 Classe Cliente 4
   - 6.2 Classe Usuário 5
   - 6.3 Classe Oportunidade 6
7. **Relacionamento entre Entidades** 7
8. **Endpoints da API** 7
   - 8.1 Endpoints de Usuários 7
   - 8.2 Endpoints de Clientes 8
   - 8.3 Enpoints de Oportunidades 9
9. **Como Funciona o Sistema?** 10
10. **Conclusão** 10

---

1. Introdução

Esta API foi desenvolvida com o objetivo de auxiliar pessoas que buscam otimizar sua mobilidade urbana, reduzir custos de deslocamento e organizar sua rotina de viagens semanais de forma eficiente.

Muitos usuários que dependem de transporte diário enfrentam dificuldades com a falta de previsibilidade, o alto custo de aplicativos de transporte convencionais e a complexidade de organizar caronas de forma recorrente. A ausência de uma ferramenta centralizada para o agendamento de trajetos pode resultar em perda de tempo, aumento de gastos e dificuldades logísticas no dia a dia.

Diante desse cenário, foi proposta a criação da API do **Bip Bip**, uma solução de caronas compartilhadas focada em agendamento inteligente e economia. A plataforma é capaz de centralizar as informações de rotas e facilitar o planejamento semanal de deslocamentos.

A API permite o registro de veículos, o gerenciamento de perfis de usuários (motoristas e passageiros) e o controle detalhado das viagens, proporcionando maior previsibilidade, segurança e uma redução significativa nos custos em relação aos concorrentes do mercado.

---

2. Objetivo da API

O principal objetivo da API do **Bip Bip** é fornecer uma solução tecnológica acessível e eficiente para que usuários possam planejar sua mobilidade urbana e reduzir custos operacionais de transporte.

Entre os principais objetivos do sistema estão:

- **Organizar a rotina de deslocamento:** Facilitar o planejamento de trajetos recorrentes através de uma agenda semanal.
- **Registrar e acompanhar viagens:** Monitorar o ciclo de vida de cada carona, desde o agendamento até a conclusão.
- **Gerenciar perfis de motoristas e passageiros:** Administrar as informações dos usuários e a disponibilidade de assentos em veículos cadastrados.
- **Otimizar o custo por trajeto:** Estruturar um modelo de compartilhamento que garanta valores inferiores aos dos concorrentes de mercado.
- **Centralizar dados logísticos:** Reunir histórico de rotas, horários e preferências em um único sistema de gestão de caronas.

Com isso, os usuários conseguem ter maior previsibilidade sobre seus horários, manter um histórico de suas viagens e economizar recursos financeiros, transformando o transporte diário em um processo mais inteligente e sustentável.

---

3. Público-Alvo

Esta API foi projetada para atender principalmente:

- **Estudantes e Trabalhadores:** Pessoas com rotinas fixas que buscam previsibilidade em seus deslocamentos diários.
- **Usuários de Aplicativos de Mobilidade:** Pessoas que desejam uma alternativa mais econômica em relação aos preços dinâmicos dos concorrentes.
- **Motoristas Particulares:** Condutores que realizam trajetos recorrentes e desejam compartilhar custos de combustível e manutenção.
- **Grupos de Carona Organizáveis:** Comunidades e coletivos que precisam de uma ferramenta centralizada para gerenciar agendas semanais.

A solução busca oferecer uma ferramenta simples e acessível para organizar o planejamento logístico semanal, garantindo o menor custo por viagem e a máxima eficiência na rotina de transporte.

---

4. Tecnologias Utilizadas

O Sistema foi desenvolvido utilizando tecnologias modernas do ecossistema backend, que permitem maior organização, escalabilidade e facilidade de manutenção.

Item	Tecnologia
Linguagem de Programação	TypeScript

Framework Backend	NestJS
ORM	TypeORM
Banco de Dados	MySQL
Documentação	Swagger
Linguagem de Consulta	SQL

O Framework (NestJS) foi utilizado para estruturar a aplicação em módulos e facilitar a organização do código, enquanto o TypeORM foi utilizado para realizar a comunicação entre a aplicação e o banco de dados.

---

5. Arquitetura da Aplicação

A aplicação foi desenvolvida seguindo uma arquitetura baseada em camadas, característica do framework NestJS.

A estrutura do sistema é composta pelos seguintes componentes:

Controllers: Responsáveis por receber as requisições HTTP e direcioná-las para os serviços correspondentes.

Services: Responsáveis pela implementação da lógica de negócio da aplicação.

Entities: Representam as tabelas do banco de dados e definem a estrutura das informações armazenadas.

ORM (TypeORM): Responsável por realizar a comunicação entre a aplicação e o banco de dados.

O fluxo básico de seu funcionamento ocorre da seguinte forma:

Cliente → Controller → Service → ORM → Banco de Dados

---

6. Estrutura do Banco de Dados

O sistema é composto por três entidades principais que estruturam a lógica de mobilidade e compartilhamento:

- Veiculo: Representa o ativo fundamental para a realização do serviço, armazenando dados como modelo, placa e capacidade de passageiros para garantir a segurança e o conforto da carona.
- Usuario: Representa tanto os motoristas quanto os passageiros, centralizando informações de perfil, preferências de viagem e autenticação para o uso da plataforma.
- Viagem: Representa a oportunidade de deslocamento, conectando as rotas de origem e destino aos horários agendados e permitindo o monitoramento do ciclo de vida de cada carona.

Essas entidades representam os elementos fundamentais de um sistema de caronas compartilhadas, permitindo registrar frotas individuais, os perfis dos participantes e o gerenciamento estratégico da agenda semanal de viagens.

6.1 Classe Veiculo

A Entidade Veículo representa os veículos disponibilizados para as viagens.

Atributo	Tipo	Descrição
id	number	Identificador único do veículo no Sistema
foto	string	URL ou endereço da foto de perfil do motorista

tipo	enum	Tipo de veículo utilizado na carona
marca	string	Marca do veículo utilizado na carona
modelo	string	Modelo do veículo utilizado na carona
corVeiculo	string	Cor do veículo utilizado na carona
placa	string	Placa do veículo utilizado na carona
viagem	Viagem[ ]	Lista de viagens associadas ao veículo

6.2 Classe Usuário

A Entidade Usuário representa os usuários que utilizam o sistema.

Atributo	**Tipo**	**Descrição**
id	number	Identificador único do usuário
nome	string	Nome completo do usuário
usuario	string	Nome de usuário utilizado para acesso ao sistema

senha	string	Senha utilizada para autenticação
foto	string	URL ou endereço da foto de perfil do usuário
viagem	Viagem[ ]	Lista de viagens associadas ao usuario

6.3 Classe Viagem

A Entidade Viagem representa as viagens registradas no sistema.

Atributo	**Tipo**	**Descrição**
id	number	Identificador único da viagem
embarque	string	Endereço do local de embarque
destino	string	Endereço do local de embarque
tempoViagem	number	Tempo estimado da viagem
distancia	number	Distância em km do trajeto da viagem
agendamento	boolean	Identifica se a viagem é para uma data futura ou atual

dataViagem	Date	Identifica a data e horário que a viagem será realizada
pagamento	enum	Identifica o meio de pagamento utilizado para a viagem
valor	number	valor da viagem
usuario	Usuario	Dados usuário relacionado a viagem
veiculo	Veiculo	Dados do veículo relacionado a viagem

---

7. Relacionamento entre Entidades

O Sistema possui relacionamentos entre as entidades para representar o fluxo de negociação.

- Um veículo pode realizar várias viagens.
- Uma viagem pode ter um veículo e um usuário.
- Um usuário pode estar em várias viagens.

Esses relacionamentos permitem acompanhar quais veículos possuem viagens cadastradas e quais usuários são responsáveis por cada viagem.

---

8. Endpoints da API

*A API foi estruturada seguindo o padrão REST, com rotas específicas para a manipulação dos recursos do sistema. Todas as operações utilizam os métodos HTTP correspondentes para garantir a integridade do ciclo **CRUD** (Criação, Consulta, Atualização e Exclusão).*

Abaixo, detalhamos os endpoints disponíveis em cada módulo:

A API segue o padrão REST, disponibilizando endpoints para manipulação dos recursos do sistema.

Os endpoints permitem realizar operações de:

- Criação de registros
- Consulta de dados
- Atualização de informações
- Exclusão de registros

Todas essas operações correspondem ao padrão CRUD (Create, Read, Update e Delete).

Veremos a seguir os exemplos de Endpoints implementados em nosso Sistema:

8.1 Endpoints/**Gerenciamento* de Usuários

Método	Endpoint	Descrição
GET	/usuarios	Retorna todos os usuários cadastrados
GET	/usuarios/{id}	Retorna um usuário específico
POST	/usuarios/logar	Autenticar o usuário no sistema
POST	/usuarios/cadastrar	Cadastra um novo usuário
PUT	/usuarios/:{id}	Atualiza os dados de um usuário

8.2 Endpoints/**Gestão* de Veiculos

Método	Endpoint	Descrição
GET	/veiculos	Lista todos os veículos
GET	/veiculos/{id}	Retorna um veículo específico
POST	/veiculos	Cadastra um novo veículo
PUT	/veiculos	Atualiza os dados de um veículo
DELETE	/veiculos/{id}	Remove um veículo do sistema

8.3 Endpoints/**Controle* de Viagens

Método	Endpoint	Descrição
GET	/viagens	Lista todas as viagens
GET	/viagens/{id}	Retorna uma viagens específica

POST	/viagens	Cria uma nova viagem
PUT	/viagens/{id}	Atualiza os dados de uma viagem
DELETE	/viagens/{id}	Remove uma viagem

9. Como Funciona o Sistema?

9.1. Gestão de Perfis e Autenticação

O sistema separa as entidades entre **Passageiros** e **Motoristas** (que podem ser o mesmo usuário em contextos diferentes).

- **Endpoints de Cadastro:** Registram dados pessoais, veículos e preferências de rota.
- **Validação:** O backend garante a segurança dos dados e a autenticação necessária para as transações de reserva.

9.2. Planejamento de Rotas e Agendamento Semanal

Diferente de apps de "corrida instantânea", o foco do Bip Bip é a **recorrência**.

- **Fluxo de Criação:** O usuário define uma origem, um destino e uma "grade horária" (ex: Segunda a Sexta, às 08h00).
- **Persistência:** O sistema armazena esses agendamentos para criar uma "viagem mestre" que se repete, facilitando a rotina do usuário sem a necessidade de solicitar uma nova carona todos os dias.

9.3. Matchmaking e Funil de Ocupação

O backend processa a compatibilidade entre quem oferece e quem busca a carona:

- **Geolocalização:** Filtra motoristas que percorrem trajetos similares nos mesmos horários.
- **Gestão de Assentos:** O sistema monitora a capacidade do veículo em tempo real. Cada vez que um passageiro confirma o agendamento, uma "vaga" é subtraída da disponibilidade do motorista para aquele trecho específico.

9.4. Ciclo de Vida da Viagem (Workflow)

O status da carona passa por etapas monitoradas pela API:

1. **Solicitada:** Aguardando confirmação do motorista ou disponibilidade de match.
2. **Confirmada:** Viagem agendada e garantida na agenda semanal.
3. **Em Andamento:** Registro do início do trajeto.
4. **Finalizada:** Conclusão que libera o processamento do pagamento e a avaliação.

9.5. Estrutura de Baixo Custo

Para garantir o diferencial competitivo de preço, o sistema utiliza:

- **Rateio Inteligente:** O backend calcula o valor com base na divisão de custos operacionais (combustível/pedágio) entre os ocupantes, em vez de tarifas dinâmicas de lucro por km rodado.
- **Otimização de Trajeto:** Agrupa o maior número de pessoas em uma única rota direta, maximizando a ocupação do veículo.

---

10. Conclusão e Considerações Finais

A API **Bip Bip** foi consolidada como uma solução estratégica para a modernização da mobilidade urbana compartilhada. O projeto cumpre o objetivo de transformar o deslocamento diário em um processo planejado e acessível, centralizando o agendamento de rotas semanais e garantindo a previsibilidade necessária para a rotina dos usuários.

Graças à robustez do **NestJS** e à tipagem segura do **TypeScript**, a aplicação entrega uma infraestrutura de backend resiliente, capaz de processar fluxos de caronas de forma eficiente e com baixo custo operacional. O resultado é um sistema que não apenas conecta motoristas e passageiros, mas estabelece uma base de dados íntegra para a gestão inteligente de trajetos recorrentes.

Próximas Evoluções

A arquitetura modular adotada permite que o **Bip Bip** escale de forma sustentável, com implementações futuras previstas para:

- **Módulos de Pagamento Integrados:** Automatização do rateio de custos entre os usuários.
- **Geolocalização em Tempo Real:** Monitoramento dinâmico das caronas em progresso.

- **Sistema de Avaliação e Gamificação:** Implementação de níveis de confiança para usuários e motoristas.
- **Controle de Acesso Avançado:** Diferenciação de perfis e autenticação multifator para maior segurança.

Com esta entrega, o sistema estabelece um alicerce sólido para uma operação de caronas compartilhadas que prioriza a economia, a organização e a eficiência de ponta a ponta.

Contatos

Sabrina Novaes - GitHub | Linkedin

Bianca Caetano - GitHub | Linkedin

Clarisse Rodrigues - GitHub | Linkedin

Gabriela Almeida - GitHub | Linkedin

Nathalia Scalercio - GitHub | LinkedIn

Leonardo Botelho - GitHub | Linkedin

Ramon Alberto - GitHub | Linkedin